



Support Vector Machines in R: Kernels, Margins, and the Cost Parameter

SVM – General Philosophy

The central philosophy of Support Vector Machines is to find the separating hyperplane that **maximizes the margin** between classes. Unlike other classifiers that simply seek any decision boundary, SVMs seek the "optimal" boundary that offers the greatest possible distance between classes.

Maximum Margin Concept

The margin is the perpendicular distance between the decision hyperplane and the nearest data points from each class. By maximizing this margin, the SVM creates the widest possible separation "corridor," which theoretically leads to better generalization on unseen data.

The data points that lie exactly on the edges of this corridor are called **support vectors** — they are the only points that actually influence the position of the decision hyperplane.

Advantages of the Approach

- **Rigorous Mathematical Foundation:** Based on convex optimization theory with strong theoretical guarantees
- **Good Generalization:** Margin maximization reduces the risk of overfitting
- **Efficiency:** Only support vectors are needed to define the final classifier
- **Extensibility:** The use of kernels allows handling non-linearly separable data

SVM – When Data Is Linearly Separable

In the simplest case, when data is linearly separable, the SVM finds the optimal hyperplane that perfectly divides the two classes while maximizing the margin between them.

Separating Hyperplane

A linear boundary (line in 2D, plane in 3D, hyperplane in n-dimensions) that divides the feature space into two distinct regions

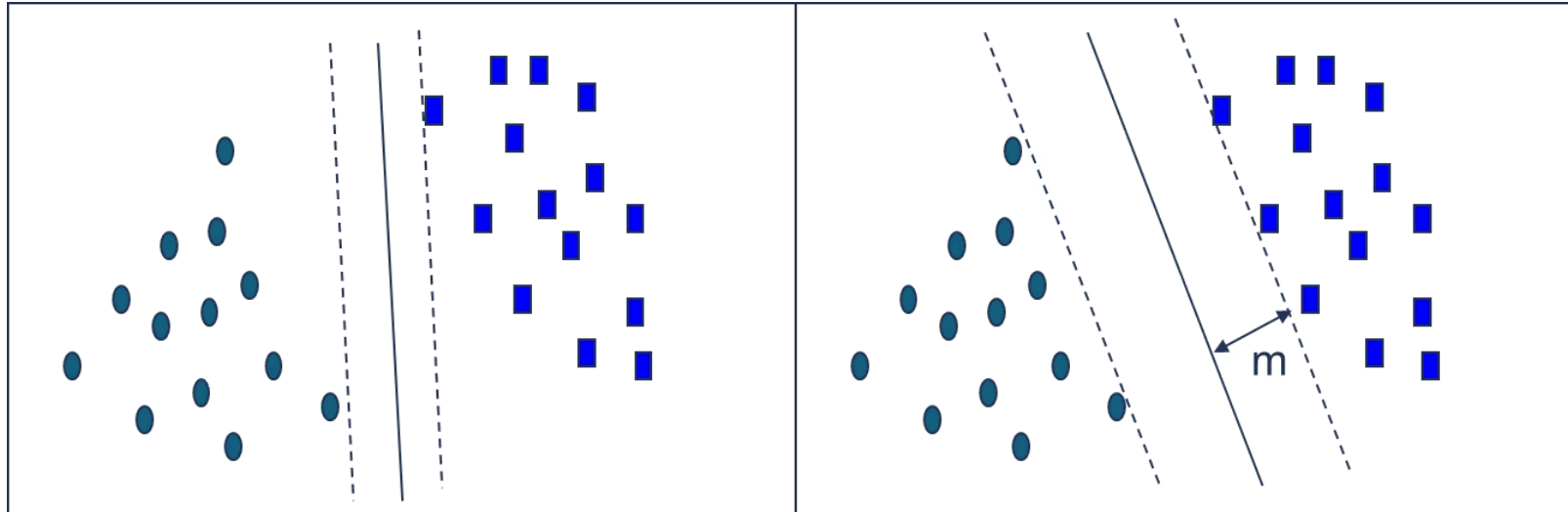
Support Vectors

Data points from both classes that are closest to the decision hyperplane and define the margins

Maximum Margin

The perpendicular distance between the hyperplane and the nearest support vectors, which is maximized during training

In the linear separability scenario, infinitely many hyperplanes can separate the classes. The SVM chooses the one with the **largest margin**, as it theoretically offers better generalization. Mathematically, this is formulated as a convex optimization problem that guarantees finding the globally optimal solution.



SVM – Linearly Separable: Mathematical Formulation

Mathematical Formulation

For linearly separable data, the decision hyperplane can be expressed as:

$$w \cdot x + b = 0$$

Where **w** is the weight vector (normal to the hyperplane) and **b** is the bias. Support vectors satisfy:

$$|w \cdot x + b| = 1$$

The total margin is then $2/||w||$, and the objective is to minimize $||w||^2$ subject to correct classification constraints.

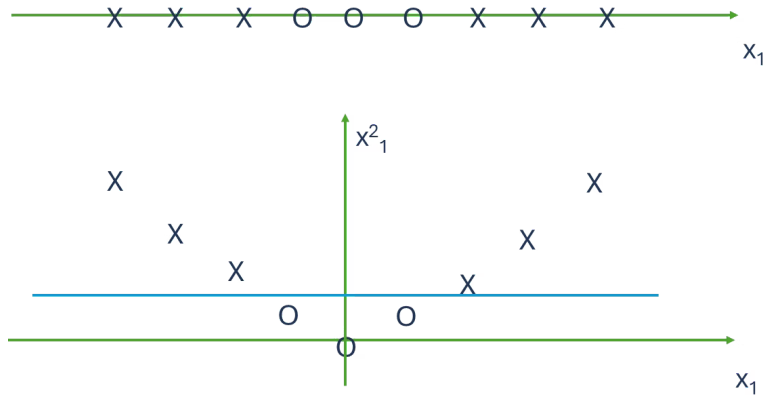
Key Properties

- **Unique Solution:** For linearly separable data, there is exactly one maximum-margin hyperplane
- **Dependence Only on Support Vectors:** Points far from the margin do not influence the solution
- **Robustness:** Small perturbations in data that do not affect support vectors do not change the classifier

□ In practice, perfectly linearly separable data is rare. That is why SVMs use techniques such as **soft margins** and **kernels** to handle more realistic scenarios.

Separating Classes with a Line

This visualization illustrates the fundamental concept of linear separation in a two-dimensional space. Points marked with **X** represent one class, while points marked with **O** represent another class.



Two-Dimensional Visualization

In the space defined by axes $\mathbf{x}_1$ and $\mathbf{x}_2$, a straight line can be drawn to completely separate the two classes. This line represents the decision hyperplane in two dimensions.

Each point in the space can be classified by determining which side of the line it falls on. Points above the line belong to one class, while points below belong to the other.

Extension to Higher Dimensions

The concept generalizes naturally:

- **2D:** Straight line
- **3D:** Plane
- **nD:** (n-1)-dimensional hyperplane

The mathematics remains consistent regardless of the number of dimensions, making SVMs scalable to problems with thousands of features.

SVM – Linearly Non-Separable Data

In most real-world problems, data is not linearly separable in the original feature space. SVMs address this through two main strategies: **soft margins** and the **kernel trick**.

Soft Margins

Allows some points to violate the margin or even be misclassified, introducing a penalty parameter **C** that controls the trade-off between maximizing the margin and minimizing classification errors. Points that violate the margin contribute to the cost function.

Kernel Trick

Implicitly maps data to a higher-dimensional feature space where linear separation becomes possible. The kernel allows computing inner products in this high-dimensional space without explicitly computing the transformed coordinates.

Non-Linear Decision Boundaries

Although the hyperplane is linear in the transformed space, it corresponds to a non-linear decision boundary in the original space. This allows SVMs to capture complex relationships between features.

- ❏ **Parameter C:** Larger values of C result in narrower margins with fewer violations (risk of overfitting), while smaller values allow wider margins with more violations (better generalization, but possible underfitting).

Kernel Functions for Non-Linear Classification

Kernel functions are at the heart of SVMs' ability to handle non-linearly separable data. They allow operating implicitly in high-dimensional spaces without the computational cost of explicitly computing the transformed coordinates.

Polynomial Kernel

$$K(x, y) = (x \cdot y + c)^d$$

Maps data to a polynomial-dimensional space. The degree **d** controls complexity. Useful when polynomial interactions between features are expected.

RBF Kernel (Gaussian)

$$K(x, y) = \exp(-\gamma \|x - y\|^2)$$

The most popular kernel. Maps to an infinite-dimensional space. The parameter γ controls the influence of each training example.

Sigmoid Kernel

$$K(x, y) = \tanh(\alpha x \cdot y + c)$$

Similar to neural networks with one hidden layer. Does not always produce a valid kernel, but can be effective in certain domains.

Choosing the Right Kernel

- **Linear:** For linearly separable or high-dimensional data
- **RBF:** A good default choice when little is known about the data
- **Polynomial:** When specific polynomial interactions are suspected

📌 **Cross-Validation:** The choice of kernel and its parameters should be made through cross-validation to avoid overfitting and ensure good generalization.

SVM – Method Summary: cla_svm

What is cla_svm?

cla_svm is the daltoolbox implementation of the Support Vector Machine classifier. It finds the maximum-margin hyperplane that best separates classes in the feature space. When data is not linearly separable, it uses kernel functions to implicitly map data to higher-dimensional spaces where linear separation becomes feasible.

Key Parameters

epsilon

Insensitive margin around the decision boundary. Points within this margin do not contribute to the loss. Default: 0.0

cost (C)

Penalty for margin violations. High C: narrow margin, fewer errors (risk of overfitting). Low C: wider margin, more violations (better generalization).

kernel

Function that defines the feature space transformation. Options: linear, radial (RBF), polynomial, sigmoid. Default: radial.

When to Use SVM?

- Data with complex, non-linear decision boundaries
- High-dimensional feature spaces (e.g., text, genomics)
- Small to medium datasets where margin maximization is beneficial
- When interpretability is less critical than predictive accuracy

Practical Considerations

- Feature scaling is essential — SVMs are sensitive to feature magnitude
- Kernel and hyperparameter selection should be done via cross-validation
- Training time grows with dataset size ($O(n^2)$ to $O(n^3)$)
- Multiclass classification is handled via one-vs-one or one-vs-rest strategies

Support Vector Machine: cla_svm

```
# Loading the package
library(daltoolbox)

# Data and levels
iris <- datasets::iris
slevels <- levels(iris$Species)

# Train/test split
set.seed(1)
sr <- sample_random()
sr <- train_test(sr, iris)
iris_train <- sr$train
iris_test <- sr$test

# Training
# epsilon: insensitive margin
# cost: penalty for violations
# default kernel; adjust as needed
model <- cla_svm("Species", sllevels,
  epsilon=0.0, cost=20.0)
model <- fit(model, iris_train)
```

```
# Evaluation — train
train_pred <- predict(model, iris_train)
train_eval <- evaluate(model,
  adjust_class_label(iris_train[, "Species"]),
  train_pred)
# accuracy: 0.975

# Evaluation — test
test_pred <- predict(model, iris_test)
test_eval <- evaluate(model,
  adjust_class_label(iris_test[, "Species"]),
  test_pred)
# accuracy: 1.000
```

Step by Step

- Use `cla_svm()` with `epsilon`, `cost`, and optionally `kernel`
- `cost`: penalty for classification errors — higher values reduce the margin
- `kernel`: linear, radial, polynomial, or sigmoid
- Training accuracy: **0.975** | Test accuracy: **1.000**
- Maximizes the margin between classes in the feature space

❏ **Key Concept:** SVM finds the **maximum-margin hyperplane** between classes. The kernel allows separating non-linearly separable classes by mapping to higher-dimensional spaces.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

